

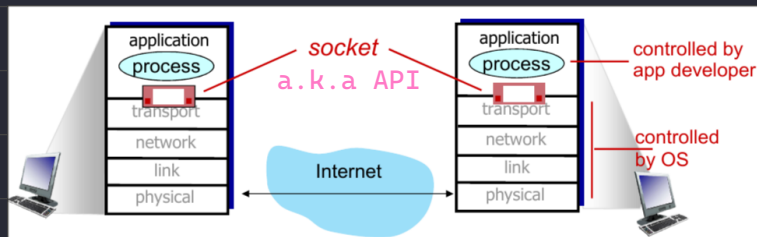
Principles of network applications

Processes communicating

process: program running within a host

1. Within same host, two processes communicate using inter-process communication (defined by OS)
2. Processes in different hosts communicate by exchanging messages

Sockets



What transport service does an app need?

Data integrity: 確保資料100%準確 e.g. file transfer

Throughput: e.g. Multi media

Timing: Low latency e.g. Online games

Security: encryption, data integrity ...

Internet transport protocols services (p 2.14)

TCP:

1. Reliable transport: 資料發生錯誤自動重傳, 確保正確
2. Flow control:

慢點送, 讓傳輸端不要發送過多data, 炸掉接收端

3. Congestion control (壅塞控制):

送慢點, 讓整體網路不要有太多的 load (diff from 2.)

(p.s. 2 與 3 的實作方法都是降低發送速度, 但是效果不同)

4. Does not provide:

即時性 (重傳)、吞吐量 (2. 3.)、安全性

5. Connection-oriented:

需先在client 與 server 建立邏輯連線, 才能提供 1~4 功能

UDP:

1. 無法保證 TCP 能提供的所有功能
2. 主要是提供一個 PORT Number 與簡單的 error check → 即時

Securing TCP



Transport Layer Security (TLS)

provides encrypted TCP connections

↻ data integrity

↻ end-point authentication

Web and HTTP

HTTP overview

`www.someschool.edu` / `someDept/pic.gif`

host name *Path Name*

HTTP uses TCP

1. client 使用 PORT 80 建立 TCP Connection
2. server 接受連線
3. HTTP messages exchanged between browser and Web server
4. TCP connection closed

HTTP is "stateless"

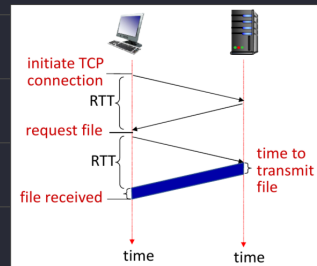
server won't store the status of the client bc it uses more resources

HTTP connections: two types

1. Non-persistent (非持續) HTTP (HTTP1.0)

→ At most one object sent over a TCP connection

- 1) Client initiates TCP connection
- 2) Server side at port 80 "accepts" the connection, notify client
- 3) Client sent requests message (involving URL) to the server
- 4) Server sent response message of the object
- 5) Server closed the TCP connection
- 6) After client recieved message, will need to start over again if response indicates that need more object



RTT (Round Trip Time): Time of a packet travel from client to server and back

HTTP response time (per object): TCP init RTT + req/resp RTT + file trans time

→ Non-persistent HTTP response time = 2RTT+ file transmission time

2. Persistent HTTP (HTTP1.1)

→ Multiple object sent and recieved on both side over a TCP connection

- 1) Server leaves connection open after sending response
- 2) Client send more requests using the previous connectin

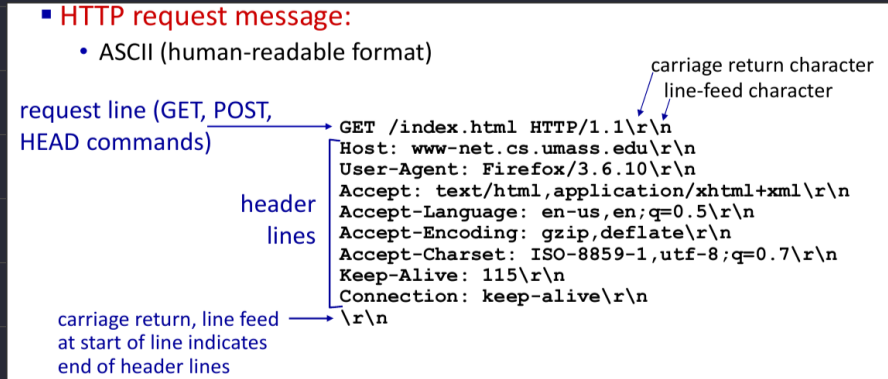
→ 省去 HTTP 1.0 的每次 requests 的 TCP establish RTT

→ HTTP 1.0 : 建立多個 TCP 連線 / HTTP 1.1 : 在一個TCP連線中併發 10 個請求 (較快)

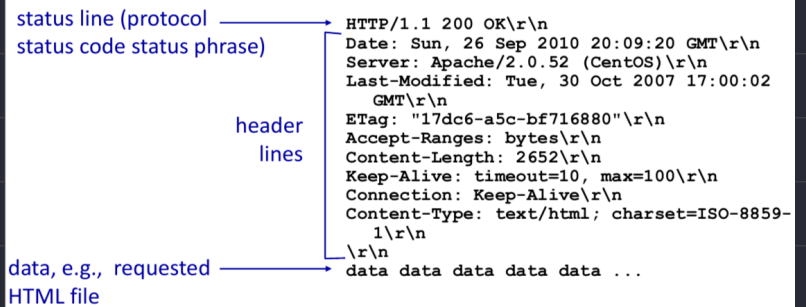
HTTP 類型：request, response

HTTP Requests Type: POST, HEAD, GET, PUT ...

HTTP Messages:



HTTP response message



HTTP Status Code:

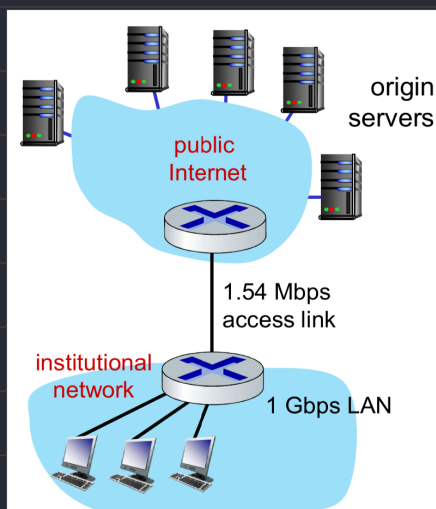
301 (Move Permanently), 400 (Bad Requests), 505 (HTTP Version Not Support)

Use cookies to ident clients:

C'uz HTTP itself is stateless

Web caches (Proxy Server):

在 Clients與Server 作中間人，把Server 的 response cache 起來，加速回應、降低頻寬佔用



加速回應的方法：

1. 提升網路速度

花錢升級 bandwidth: 把 1.54Mbps → 1.54Gbps

2. 安裝 Web Cache

將資料 cache 在 local network 的 Server

Web caches (proxy servers)

To speed up the site loading speed, we have two options

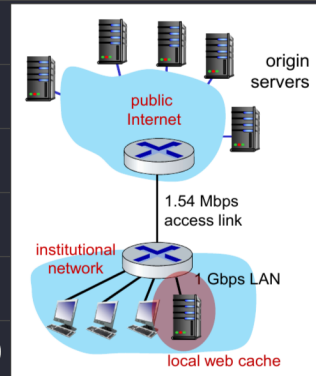
(1) Speed up the access link to the server → Expensive

(2) Install a web cache → Cheap

If some resource is available on cache server:

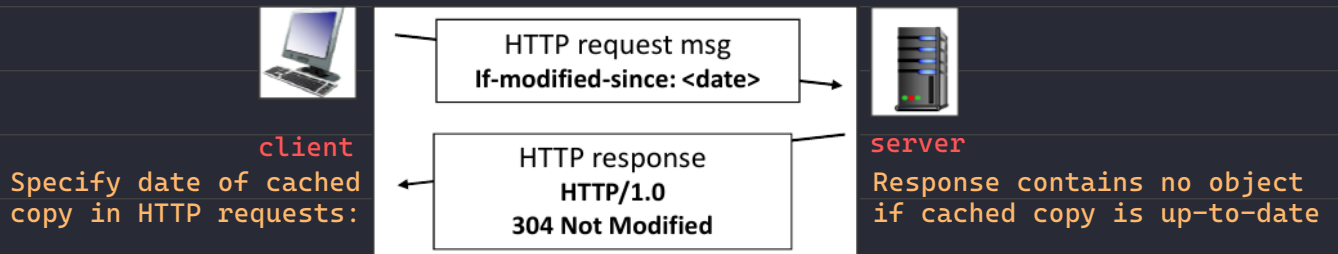
Original speed: $100\% * (\text{access link speed})$

After speed: $60\% * (\text{LAN speed}) + 40\% * (\text{access link spd})$



Conditional GET

Don't send object if cache has up-to-date cached version

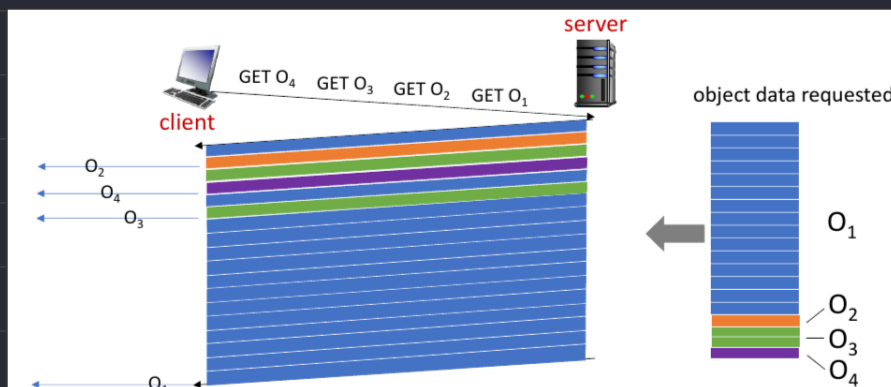


HTTP 2

Cons of HTTP 1

Server follow FCFS(first-come-first-served), leading HOL(head-of-line blocking)

Most of the object must wait until other object's transmission time



HTTP 3

HTTP2: No security over TCP, 從封包遺失中恢復的過程仍然會導致所有物件的傳輸停滯

Add security, per object error control over UDP

E-mail, SMTP, IMAP

Three major components:

✧ User agents

Mail reader e.g. Outlook, Thunderbird

✧ Mail servers

mailbox: contains incoming messages for user

message queue of outgoing (to be sent) mail messages

✧ Simple mail transfer protocol: SMTP

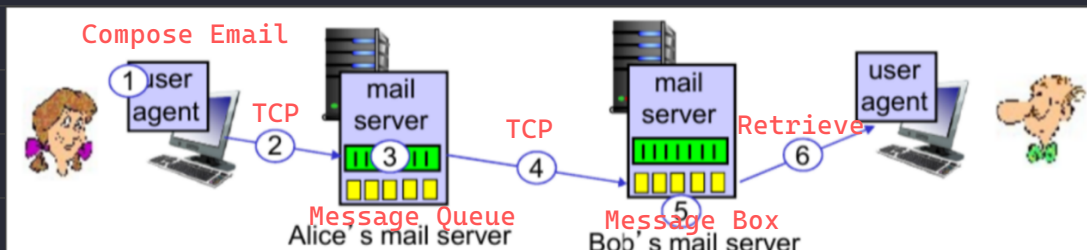
client: sending mail server

server: receiving mail server

E-mail: the RFC (5321)

- ✧ uses TCP port 25 to reliably transfer email message from client to server,
- ✧ direct transfer: sending server (acting like client) to receiving server
- ✧ three phases of transfer
 - handshaking (greeting)
 - transfer of messages
 - closure
- ✧ command/response interaction (like HTTP)
 - commands: ASCII text
 - response: status code and phrase
- ✧ messages must be in 7-bit ASCII

Scenario: Alice sends e-mail to Bob



Comparison with HTTP

HTTP

pull

< both have ASCII cmd / response / status >

Each object has its own response msg

SMTP

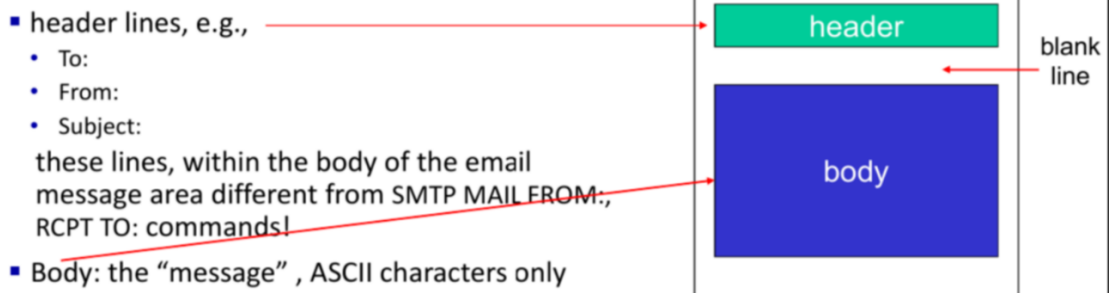
push

Mutiple object in one message

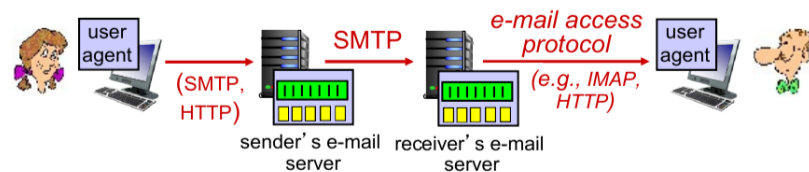
Uses persistent connection

Require 7-bit ASCII header & body

SMTP protocol defined in RFC 822



Mail access protocols

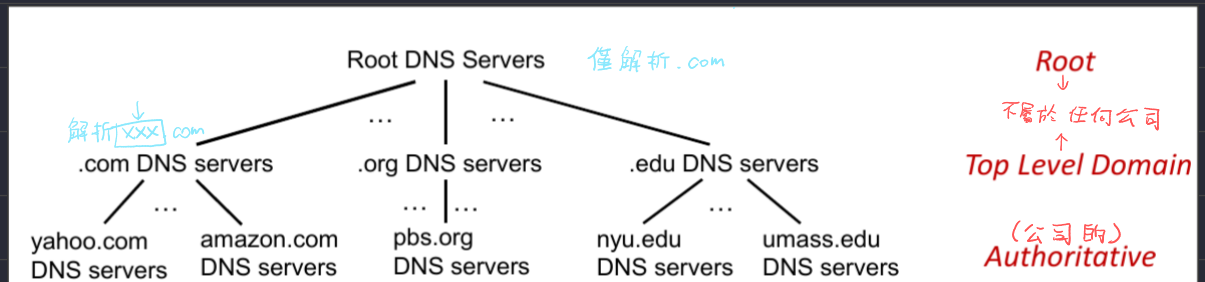


- **SMTP**: delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
 - **IMAP**: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP**: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

The Domain Name System (DNS)

DNS: Hostname → IP Address

→ distributed database implemented in hierarchy of many name servers



13 logical root name servers worldwide, each sever replicated many times

→ application-layer protocol: hosts, name servers communicate to resolve names

Top-Level Domain (TLD) servers

responsible for .com, .org, .net/ top-level country domains, e.g.: .ca, .jp

Network Solutions: authoritative registry for .com, .net TLD

Authoritative DNS servers:

organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts (e.g. 北科 CNC)

Local DNS name servers

→ each ISP has one (a.k.a default name server)

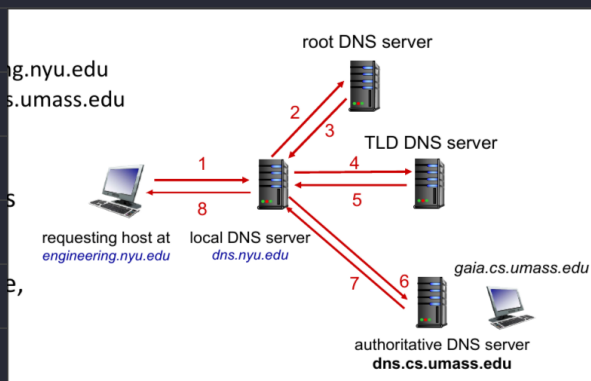
→ when host makes DNS query, query is sent to its local DNS server

1. has local cache of recent name-to-address translation pairs

2. acts as proxy, forwards query into hierarchy

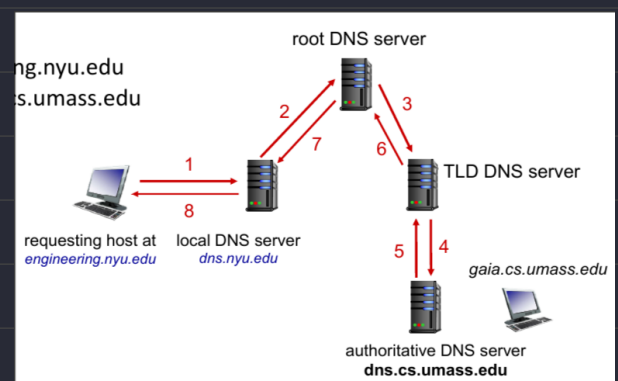
DNS name resolution → Userspace 不會知道採用哪種方式

1. iterated query (迭代式)



比較常見

2. recursive query (遞回式)



Cons: 越上層的負擔越重

Caching, Updating DNS Records

每個 Server 都會 cache the result of DNS resolution

cache entries timeout (disappear) after some time (TTL (Time to live))

cached entries may be out-of-date: cache 來不及更新

DNS records

RR format: (name, value, type, ttl)

type=A:

name: Hostname

value: IP address

type=NS:

name: domain (google.com)

value: hostname of authoritative name server

type=CNAME:

name: alias name

value: real name

e.g. www.bmi.com → servereast.backup2.ibm.com

type=MX

name: name of mail services

value: mail server's domain

Inserting records into DNS

(networkutopia.com, dns1.networkutopia.com, NS)

(dns1.networkutopia.com, 212.212.212.1, A)

P2P applications

P2P: 同時當 server 與 client, 允許 end devices 間互相傳輸

Peer-to-peer (P2P) architecture

The role of clients and servers changes depend on requirements

Any end system can communicate directly

The file you requests may have copy an other clients, the speed could be fast

Self scalability: new peers = + service capacity = + services demand

File distribution: client-server vs P2P

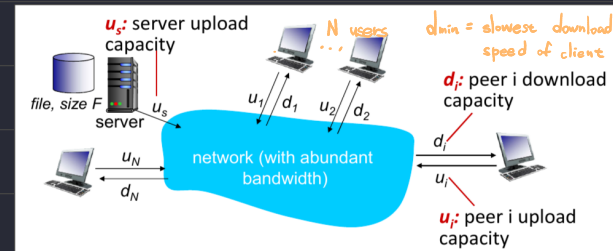
Client Server

$$\text{Server upload: } \frac{NF}{u_s}$$

$$\text{Client download: } \frac{F}{d_{\min}}$$

$$\rightarrow \text{Speed} = D_{c-s} \geq \max\left(\frac{NF}{u_s}, \frac{F}{d_{\min}}\right)$$

$\rightarrow$ When N goes too large, Server will slow down whole speed



P2P

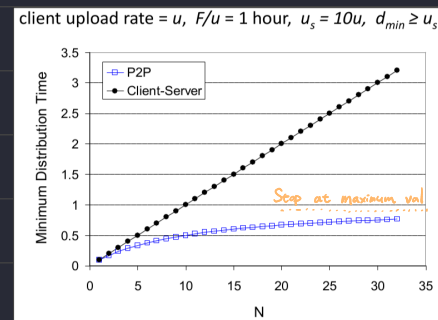
$$\text{Server transmission: } \frac{F}{u_s} \quad (\text{Send at least one copy})$$

$$\text{Client download: } \frac{F}{d_{\min}}$$

$$\text{Clients upload: } u_s + \sum_{i=1}^N u_i$$

$$\rightarrow \text{Speed} = D_{P2P} \geq \max\left(\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{(u_s + \sum u_i)}\right)$$

$\rightarrow$ When N goes too large, Server load not goes on linearly



P2P file distribution: BitTorrent

Tracker: Tracks peers participating in torrent

Torrent: Group of peers exchanging chunks of a file

churn: P2P 螞蟥, 只享受利益不負擔義務

BitTorrent: requesting, sending file chunks

How to design a good P2P system ?

1. Resuts the rarest chunk first: 檢查所有人中對少持有的chunk優先索取

2. Send data to the client that send the chunk at highest rate first: 先給上傳高的

$\rightarrow$ tit-for-tat: 互惠原則

$\rightarrow$ Evevry 30s, randomly select another peer, stats send chunks

(optimistically unchoke)

Video streaming and content distribution networks (CDN)

Combine client-server and P2P architecture

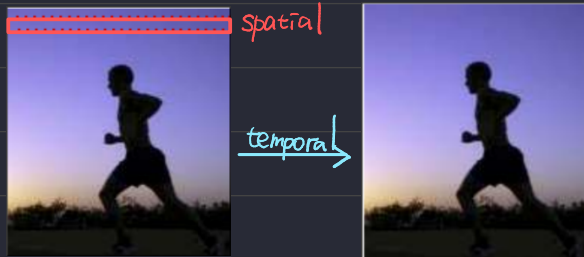
solution: distributed, application-level infrastructure

Multimedia: video

coding: use redundancy within and between images to decrease # bits

spatial (within image) : Sending a row of purple instead of purple x n times

temporal (from one image to next): Only send difference between two frames



CBR (Constant bit rate)

VBR (Variable bit rate)

Depends on rate changes as amount of spatial, temporal

Streaming stored video

Def: client playing out early part of video,
while server still sending later part of video

Challenges:

1. server-to-client bandwidth will vary over time, with changing network congestion levels
2. packet loss and delay due to congestion will delay playout, or result in poor video quality
3. continuous playout constraint: once client begins, playback must match original timing
→ but network delays are variable (jitter), so we use client-side buffer
4. Client side Fast Forward the video
5. Video packet lost and needed to be retransmit

Solution: DASH (Dynamic, Adaptive Streaming over HTTP)

Server : 1. Divide video into multiple chunks

2. Encode each chunk at different rate (1080p, 720p)

3. Provide manifest file to tell client where to get chunks

Client : 1. Measure server-to-client bandwidth

2. Consulting manifest, requests one chunk at a time

→ Streaming video = encoding + DASH + playout buffering

Content distribution networks (CDNs)

How to stream content to hundreds of thousands of simultaneous users?

1. (X) Single large "MEGA-SERVER" Doesn't scale
2. (O) Store/serve multiple copies of videos at multiple geographically distributed sites (CDN)

CDN: Stores copies of content at CND nodes around the world

Client: Requests content from CDN, with the nearest and fastest one

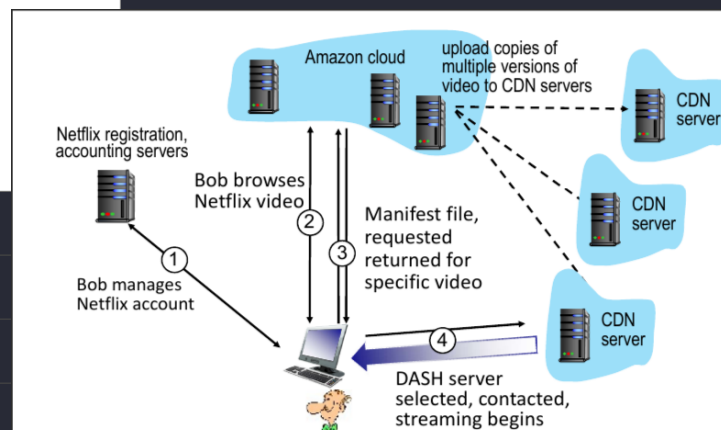
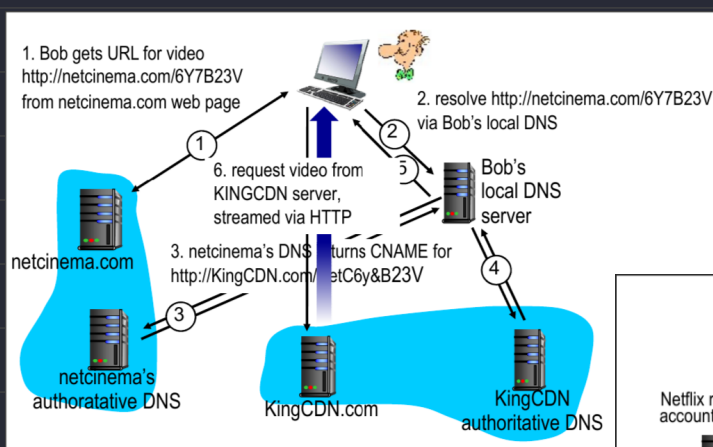
→ Choose different server if network path congested (擁擠)

OTT challenges: coping with a congested Internet



1. From which CDN node to retrieve content?
2. Viewer behavior in presence of congestion?
3. What content to place in which CDN node?

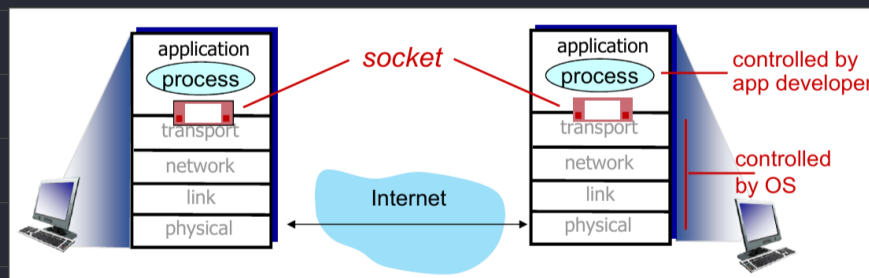
CDN content access: a closer look



Socket programming with UDP and TCP

Socket programming (Briefly)

Socket: door between application process and end-end-transport protocol



Two socket types for two transport services:

UDP: unreliable datagram

→ Fast: No need to establish the connection

TCP: reliable, byte stream-oriented